Manipulação de Arquivos em TypeScript

1. **abrir o arquivo**
2. **gravar strings**
3. **ler o conteúdo textual inteiro**
4. **separar em linhas**
5. quando necessário, **alterar o array de linhas e regravar o arquivo**

As APIs síncronas do módulo `node:fs` bloqueiam a execução até o término da operação e lançam exceções imediatamente, razão pela qual o uso de `try/finally` é o padrão mais seguro.

```
import * as fs from 'node:fs'
```

1. Abrir arquivo

A função `fs.openSync(path[, flags[, mode]])` retorna um **descritor de arquivo** numérico (`fd`). Se o `flags` não for informado, o padrão é `'r'` . Entre as flags mais úteis para texto estão: `'r'` para leitura, `'w'` para escrita com truncamento/criação, `'a'` para anexar ao fim, e `'w+'` para leitura e escrita com criação/truncamento.

Exemplos:

```
const readFd = fs.openSync('notes.txt', 'r')
const writeFd = fs.openSync('notes.txt', 'w')
const appendFd = fs.openSync('notes.txt', 'a')
```

2. Gravar uma linha

Para texto, a função mais conveniente é `fs.writeSync(fd, string[, position[, encoding]])` . Ela grava a string no arquivo, usa `'utf8'` por padrão e retorna a quantidade de bytes gravados. Para gravar uma linha, basta escrever o texto seguido de `\n` .

```
const fd = fs.openSync('notes.txt', 'a')

try {
  fs.writeSync(fd, 'First line\n', null, 'utf8')
  fs.writeSync(fd, 'Second line\n', null, 'utf8')
} finally {
  fs.closeSync(fd)
}
```

3. Ler linhas

Para leitura de arquivo texto, a forma mais simples é usar `fs.readFileSync(path[, options])` com `encoding: 'utf8'`. Quando a codificação é informada, a função retorna uma **string**; sem codificação, retorna um `Buffer`.

Depois disso, em TypeScript, separam-se as linhas com `split`.

```
const text = fs.readFileSync('notes.txt', { encoding: 'utf8', flag: 'r' })
const lines = text.split(/\r?\n/)
```

Como muitos arquivos terminam com quebra de linha, é comum remover a última linha vazia, quando ela existir:

```
const text = fs.readFileSync('notes.txt', { encoding: 'utf8', flag: 'r' })
const lines = text.split(/\r?\n/)

if (lines[lines.length - 1] === '') {
  lines.pop()
}
```

4. Fechar arquivo

A função `fs.closeSync(fd)` fecha o descritor e retorna `undefined`.

```
fs.closeSync(fd)
```

5. Observação importante sobre “alterar uma linha”

Em arquivo texto, linhas têm tamanho variável. Por isso, para um tutorial básico, a forma mais limpa de “editar uma linha” costuma ser:

- ler o arquivo inteiro
- transformar em array de linhas
- alterar a linha desejada no array
- gravar o arquivo inteiro novamente

Didaticamente, essa abordagem é muito melhor do que tentar reposicionar manualmente o cursor em bytes.

Abaixo estão três funções simples e úteis.

```typescript
import * as fs from 'node:fs'

export function appendLine(filePath: string, line: string): void {
  const fd = fs.openSync(filePath, 'a')

  try {
    fs.writeSync(fd, `${line}\n`, null, 'utf8')
  } finally {
    fs.closeSync(fd)
  }
}

export function readLines(filePath: string): string[] {
  const text = fs.readFileSync(filePath, { encoding: 'utf8', flag: 'r' })

  const lines = text.split(/\r?\n/)

  if (lines.length > 0 && lines[lines.length - 1] === '') {
    lines.pop()
  }

  return lines
}

export function replaceLine(
  filePath: string,
  lineIndex: number,
  newLine: string
): void {
  const lines = readLines(filePath)

  if (lineIndex < 0 || lineIndex >= lines.length) {
    throw new Error('Invalid line index')
  }

  lines[lineIndex] = newLine

  const fd = fs.openSync(filePath, 'w')

  try {
    fs.writeSync(fd, `${lines.join('\n')}\n`, null, 'utf8')
  } finally {
    fs.closeSync(fd)
```

```
    }
  }
```

Nesse pequeno conjunto:

- `appendLine` abre em modo de anexação e grava uma nova linha
- `readLines` lê o texto inteiro e devolve um array de linhas
- `replaceLine` modifica uma linha específica e reescreve o arquivo inteiro

As operações de abertura, escrita, leitura textual e fechamento estão de acordo com as APIs síncronas oficiais do `node:fs` .

Exemplo completo envolvendo tudo

```typescript
import * as fs from 'node:fs'

function appendLine(filePath: string, line: string): void {
  const fd = fs.openSync(filePath, 'a')

  try {
    fs.writeSync(fd, `${line}\n`, null, 'utf8')
  } finally {
    fs.closeSync(fd)
  }
}

function readLines(filePath: string): string[] {
  const text = fs.readFileSync(filePath, { encoding: 'utf8', flag: 'r' })

  const lines = text.split(/\r?\n/)

  if (lines.length > 0 && lines[lines.length - 1] === '') {
    lines.pop()
  }

  return lines
}

function replaceLine(filePath: string, lineIndex: number, newLine: string): void {
  const lines = readLines(filePath)

  if (lineIndex < 0 || lineIndex >= lines.length) {
    throw new Error('Invalid line index')
  }

  lines[lineIndex] = newLine

  const fd = fs.openSync(filePath, 'w')

  try {
    fs.writeSync(fd, `${lines.join('\n')}\n`, null, 'utf8')
  } finally {
    fs.closeSync(fd)
  }
}

const filePath = 'log.txt'
```

```
// Optional: start from a clean file
const resetFd = fs.openSync(filePath, 'w')
fs.closeSync(resetFd)

// Write lines
appendLine(filePath, 'Alice')
appendLine(filePath, 'Bob')
appendLine(filePath, 'Carol')

// Read lines
const beforeUpdate = readLines(filePath)
console.log('Before update:', beforeUpdate)

// Replace one line
replaceLine(filePath, 1, 'Beatriz')

// Read again
const afterUpdate = readLines(filePath)
console.log('After update:', afterUpdate)
```

Saída esperada:

```
Before update: [ 'Alice', 'Bob', 'Carol' ]
After update: [ 'Alice', 'Beatriz', 'Carol' ]
```

Em síntese, para **arquivos texto** em modo **síncrono**, o núcleo prático é este:

- `openSync(..., 'a')` para acrescentar linhas
- `writeSync(fd, line + '\n', ...)` para gravar linha
- `readFileSync(..., { encoding: 'utf8' })` para obter o texto
- `split(/\r?\n/)` para obter as linhas
- `openSync(..., 'w')` + `writeSync(...)` para regravar o arquivo após editar linhas
- `closeSync(fd)` para encerrar corretamente o descritor

Posso agora converter esse conteúdo em um formato de **material de aula**, com enunciado, explicação e exemplo progressivo.